2026-2027 GÜZ

C#

Nesneye Dayalı Programlama

Hafta 1 | Nesnelerle düşünme ve ilk C# uygulamaları

Prof. Dr. Mehmet BAKIR
Bilgisayar Mühendisliği Bölümü

3 saat teori + 1 saat laboratuvar

.NET 10 LTS • C# 14 • Visual Studio 2026

YENİ DERS TASARIMI

Bu kez öğrenci yalnız dinlemeyecek

ESKİ SORUN	Arka arkaya kavram, az kod teması
YENİ RİTİM	10-12 dakika anlatım, hemen ardından uygulama
ÖĞRENCİ EYLEMİ	Tahmin et → çalıştır → değiştir → açıkla
HAFTA SINIRI	if/else, döngü ve ayrıntılı sınıf yazımı sonraki haftalarda

Ders sonunda öğrenci ne yapabilecek?

- Sınıf, nesne, durum ve davranışı günlük bir problemde ayırmak
- C#, .NET ve geliştirme ortamının rollerini açıklamak
- Sıfırdan Console App oluşturup çalıştırmak
- Değişken, giriş-çıkış, null güvenliği ve TryParse kullanmak
- En az beş küçük uygulamayı değiştirerek test etmek

3+1 akışı

1. saat: Nesnelerle düşünme
2. saat: C# ve ilk program
3. saat: Veri, giriş ve hesap
+1: Rehberli laboratuvar

BAŞLANGIÇ YOKLAMASI

Kod yazmadan önce düşünelim

ÖNCE TAHMİN ET

Bir öğrenci otomasyonunda hangi şeyler veri, hangi şeyler işlem olur?

En az 3 veri + 3 işlem yazın.

Çalışma biçimi

4 dakika
1. Tek başına düşün
2. Yanındakiyle karşılaştır
3. Kodu/cevabı test et

Nesnelerle düşünmek

Önce gerçek problemi parçalayacağız; sonra C# sözdizimine geçeceğiz.

Üniversite otomasyonunda neler var?

İSİMLER	FİİLLER
Öğrenci Ders Öğretim elemanı Kayıt	Derse kaydol Not gir Ders aç Bilgi göster

Başlangıç ipucu: isimler çoğu zaman nesne/sınıf; fiiller davranış adayıdır.

Sınıf şablondur, nesne gerçek örnektir

SINIF	NESNE
Ogrenci Hangi özellikler ve davranışlar var?	ayse Ad = Ayşe No = 20260042

Bir sınıftan çok sayıda nesne üretilebilir.

Her nesne kendi durumunu taşır

ayse : Ogrenci

Ad	Ayşe Yılmaz
Numara	20260042
Ortalama	3.18
Aktif	true

Durum = nesnenin o andaki veri değerleri

Durum + davranış birlikte anlam kazanır

DURUM	DAVRANIŞ
Ad Numara Ortalama Aktif	DerseKaydol() NotGuncelle() BilgiGoster()

UYGULAMA 1

Hangisi sınıf, nesne, durum, davranış?

ÖNCE TAHMİN ET

Otomobil • benimOtomobilim • renk • hizlan()

Her ifadeyi doğru kategoriye yerleştirin.

Çalışma biçimi

3 dakika
1. Tek başına düşün
2. Yanındakiyle karşılaştır
3. Kodu/cevabı test et

Her ayrıntıyı modele taşımayız

Gerçek öğrenci

Boy, göz rengi, kahve tercihi, adres, dersler, notlar...

Ders kayıt modeli

Numara, ad, kayıtlı dersler ve gerekli davranışlar

Soyutlama: amaç için gerekli özellikleri seçmek, gereksiz ayrıntıyı dışarıda bırakmak.

Nesne kendi geçerli durumunu korur



Veri rastgele değil, tanımlı davranışlarla değişir.

Kalıtım ortak özellikleri yeniden kullanır

Kisi

Ad • E-posta

Ogrenci

Personel

Bu hafta yalnız fikir; ayrıntılı kod ilerleyen haftada.

Çok biçimlilik: aynı istek, farklı davranış

Daire.Ciz()	Daire çizer
Kare.Ciz()	Kare çizer
Ucgen.Ciz()	Üçgen çizer

Çağrı aynı: Ciz() • Sonuç nesnenin türüne göre değişir.

UYGULAMA 2

Nesne dedektifi

ÖNCE TAHMİN ET

Bir kütüphane sistemi seçin:

2 sınıf • her sınıf için 3 özellik • 2 davranış • 1 gerçek nesne

Çalışma biçimi

6 dakika
1. Tek başına düşün
2. Yanındakiyle karşılaştır
3. Kodu/cevabı test et

KONTROL NOKTASI

Bir dakikada açıklayabiliyor musunuz?

- Ogrenci neden sınıf, ayse neden nesnedir?
- Ortalama neden durumdur?
- NotGuncelle() neden davranıştır?
- Soyutlama ile kapsülleme arasındaki fark nedir?
- Kalıtım ve çok biçimlilik bu hafta neden yalnız ön izlemedir?

2. DERS

C#, .NET ve ilk çalışan program

Öğrenci artık kavramı ekranda çalışan küçük bir ürüne dönüştürecek.

Üç araç, üç farklı görev

C#	Kod yazdığımız programlama dili
.NET	Çalışma zamanı, SDK ve sınıf kitaplıkları
Visual Studio	Kod yazma, derleme, çalıştırma ve hata ayıklama ortamı

Kaynak kod makineye nasıl ulaşır?

1

Program.cs

2

C# derleyicisi

3

IL + metadata

4

CLR / JIT / AOT

Kaynak kod → derleme → ara temsil → çalıştırma

Bu dönemin kararlı çalışma tabanı

.NET 10 LTS

Kasım 2028'e kadar desteklenen uzun süreli destek sürümü.

C# 14

.NET 10 ile kullanılan kararlı dil sürümü. Yeni özellikleri ilerleyen haftalarda seçerek göreceğiz.

Visual Studio 2026

Community sürümü ders uygulamaları için yeterlidir.

İlk haftada hedef sürüm özelliklerini ezberlemek değil, temel kodu güvenle çalıştırmaktır.

İlk Console App'i birlikte oluşturalım

1	Create a new project
2	Console App • C#
3	Proje adı: Hafta1IlkProgram
4	Framework: .NET 10.0 (LTS)
5	Ctrl + F5 ile çalıştır

Modern başlangıç kodu daha az yük taşır

```
Console.WriteLine("Merhaba, C#!");
```

Top-level statements

```
using System;

class Program
{
    static void Main()
    {
        Console.WriteLine("Merhaba, C#!");
    }
}
```

Klasik Main yapısı

İkisi de aynı giriş noktası fikrine sahiptir; derste modern şablonu kullanacağız.

Tek satırı parçalara ayıralım

```
Console.WriteLine("Merhaba, C#!");
```

Console	Konsol ile çalışmayı sağlayan sınıf
.	Üyeye erişim
WriteLine	Ekrana yazan metot
(...)	Metoda verilen bilgi
;	İfadenin sonu

UYGULAMA 4

Çalıştır, sonra iki şeyi değiştir

```
Console.WriteLine("Merhaba, C#!");
Console.WriteLine("Nesnelerle düşünmeye başlıyorum.");
```

Görev

1. İlk metni adınızla değiştirin.
2. Üçüncü bir satır ekleyin.
3. Noktalı virgülü silip hata mesajını okuyun.

Write ve WriteLine aynı görünmez

```
Console.Write("Ad: " );
Console.WriteLine("Ayşe");
Console.WriteLine("Hoş geldin!");
```

Tahmin

Ekranda kaç satır oluşur?

Write imleci aynı satırda bırakır. WriteLine yazdıktan sonra alt satıra geçer.

Beklenen çıktı: Ad: Ayşe ↵ Hoş geldin!

Derleyici bize ne anlatıyor?

```
Console.WritLine("Merhaba")
console.WriteLine("C#");
```

Üç hata

- Metot adı yanlış
- Noktalı virgül eksik
- Console büyük harfle başlamalı

Hata mesajının ilk satırını ve işaret edilen satırı birlikte okuyun.

Veri, giriş ve küçük hesaplar

Şimdi program sabit metin yazmak yerine kullanıcıyla konuşacak.

Değişken etiketli bir bellek alanıdır

yas

20

Tür

int — hangi değerlerin tutulabileceğini belirler

Ad

yas — kodda değere nasıl ulaşacağımızı belirler

Değer

20 — o anda saklanan veridir

Doğru türü verinin anlamına göre seçin

string	"Ayşe" — metin
int	20 — tamsayı
double	3.18 — genel ondalıklı sayı
decimal	125.50m — parasal/hassas ondalık
bool	true — doğru/yanlış
char	'A' — tek karakter

Tanımlama ve atama farklı işlemlerdir

```
int yas;        // tanımlama
yas = 20;       // atama

string ad = "Ayşe";  // tanımlama + ilk değer
yas = 21;            // değer değişti
```

Oku

int yas; → yas adlı int değişken oluştur

İzle

yas 20 iken son satırdan sonra 21 olur

var türü kaldırmaz, derleyiciye buldurur

```
var bolum = "Bilgisayar Mühendisliği"; // string
var sinif = 1;                         // int
var ortalama = 3.18;                   // double
```

Kullan

Sağ taraftan tür açıkça anlaşılıyorsa okunabilir olabilir.

Unutma

var dinamik tür değildir; tür derleme zamanında belirlenir.

Tırnak ve son ekler türü değiştirir

`"A"`	string: çift tırnak
`'A'`	char: tek tırnak
`3.14`	double: varsayılan ondalık
`3.14m`	decimal: m son eki
`true`	bool: tırnaksız anahtar sözcük

İyi ad, yoruma duyulan ihtiyacı azaltır

```
int ogrenciSayisi = 143;
string adSoyad = "Ayşe Yılmaz";
double genelNotOrtalamasi = 3.18;

// Kaçının:
int x1 = 143;
string a = "Ayşe Yılmaz";
```

Değişken

camelCase
ogrenciSayisi

Sınıf / metot

PascalCase
OgrenciKaydi • BilgiGoster

Kural

Boşluk yok; sayı ile başlayamaz; anahtar sözcük doğrudan kullanılamaz.

String interpolation çıktıyı okunur kılar

```
string ad = "Ayşe";
int yas = 20;
double ortalama = 3.184;

Console.WriteLine($"Ad: {ad}, Yaş: {yas}");
Console.WriteLine($"GNO: {ortalama:F2}");
```

$ işareti

{ } içindeki değişkenleri metne yerleştirir.

F2

Ondalıklı değeri iki basamakla gösterir: 3.18

UYGULAMA 5

Kişisel bilgi kartını üret

```
string ad = "Kendi adınız";
string bolum = "Bilgisayar Mühendisliği";
int sinif = 1;
double ortalama = 3.25;

Console.WriteLine("--- ÖĞRENCİ KARTI ---");
Console.WriteLine($"{ad} | {bolum}");
Console.WriteLine($"Sınıf: {sinif} | GNO: {ortalama:F2}");
```

Değiştir

Kendi bilgilerinizi yazın.
Ortalama için F1 ve F3 deneyin.
Yeni bir bool değişken ekleyin.

TAHMİN ET → ÇALIŞTIR

Çıktı tam olarak ne olur?

```
int a = 5;
int b = 2;
Console.WriteLine($"{a} + {b} = {a + b}");
Console.WriteLine("{a} + {b}");
```

Dikkat

İkinci satırda $ yok. Süslü parantezler değişken yerine düz metin olarak yazılır.

Cevabı görmeden önce kâğıda yazın.

ReadLine kullanıcıdan bir satır metin okur

```
Console.Write("Adınız: " );
string? ad = Console.ReadLine();
Console.WriteLine($"Merhaba {ad}!");
```

Akış

1. İstem yazılır
2. Kullanıcı Enter'a basar
3. Metin ad değişkenine gelir
4. Kişiselleştirilmiş çıktı oluşur

ReadLine neden string? döndürür?

string	string?	null
Her zaman metin var	Metin veya null olabilir	Değer yok

? işareti bu değişkenin null olabileceğini açıkça belirtir.

Null-coalescing operatörü yedek değer seçer

```
Console.Write("Adınız: " );
string ad = Console.ReadLine() ?? "Misafir";
Console.WriteLine($"Merhaba {ad}!");
```

Sol değer null değilse

Kullanıcının girdiği metin kullanılır.

Sol değer null ise

Sağdaki "Misafir" değeri kullanılır.

?? yalnız null durumunu ele alır; boş metin ayrı bir durumdur.

Konsoldan gelen metni sayıya çevirmek gerekir

int.Parse(metin)	Geçersiz metinde exception; girdi kesin geçerliyse
Convert.ToInt32(metin)	Dönüşüm yardımcı metodu; null davranışı farklı
int.TryParse(metin, out sayi)	Başarıyı bool ile bildirir; kullanıcı girdisinde güvenli tercih

TryParse iki bilgi üretir

```
string? giris = "20";
bool basarili = int.TryParse(giris, out int yas);

Console.WriteLine($"Başarılı mı? {basarili}");
Console.WriteLine($"Yaş: {yas}");
```

Girdi: "20"

basarili = true
yas = 20

Girdi: "yirmi"

basarili = false
yas = 0

Bu hafta sonucu gözlüyoruz; true/false'a göre karar vermeyi 2. haftada if ile yapacağız.

UYGULAMA 6

Yaş okuyucu: geçerli ve geçersiz girdiyi test et

```
Console.Write("Yaşınız: " );
string? giris = Console.ReadLine();
bool basarili = int.TryParse(giris, out int yas);

Console.WriteLine($"Dönüşüm: {basarili}");
Console.WriteLine($"Gelecek yıl: {yas + 1}");
```

Üç test

20
yirmi
2147483648

Her testte iki değişkenin değerini not edin.

Kodun durumunu satır satır takip edin

Başlangıç	giris = ? • basarili = ? • yas = ?
ReadLine sonrası	giris = "20"
TryParse sonrası	basarili = true • yas = 20
Son çıktı	Gelecek yıl: 21

Breakpoint + Step Over + Locals: tahmininizi programın gerçek durumu ile karşılaştırır.

Operatör önceliği sonucu değiştirir

```
double kisa = 3;
double uzun = 5;

double alan = kisa * uzun;
double cevre = 2 * (kisa + uzun);

Console.WriteLine($"Alan: {alan}");
Console.WriteLine($"Çevre: {cevre}");
```

Önce

Parantez

Sonra

* ve /

Sonra

+ ve -

UYGULAMA 7

Dikdörtgen hesaplayıcı

```
Console.Write("Kısa kenar: " );
double.TryParse(Console.ReadLine(), out double kisa);
Console.Write("Uzun kenar: " );
double.TryParse(Console.ReadLine(), out double uzun);

double alan = kisa * uzun;
double cevre = 2 * (kisa + uzun);
Console.WriteLine($"Alan: {alan:F2}");
Console.WriteLine($"Çevre: {cevre:F2}");
```

Değiştir

Birim ekleyin.
Üç farklı ölçü deneyin.
Negatif girişte çıkan sonucu tartışın.
Çözümü 2. haftaya bağlayın.

Her hata aynı aşamada oluşmaz

Derleme hatası	Eksik ;, yanlış ad, tür uyuşmazlığı
Çalışma zamanı hatası	Program çalışırken oluşan exception
Mantık hatası	Program çalışır ama yanlış sonuç üretir

İyi hata ayıklama sorusu: 'Programın hangi satırda hangi değeri taşımasını bekliyordum?'

HATA AVI 2

Bu programda üç farklı sorun var

```
Console.Write("Yaş: " )
int yas = Console.ReadLine();
Console.WriteLine("Gelecek yıl: " + yas + 1);
```

Bul

1. Derleme işareti
2. Tür uyuşmazlığı
3. Birleştirme / hesaplama sorunu

Düzeltme hedefi: ReadLine → TryParse → yas + 1 → interpolation

+1 LAB | ANA GÖREV

Öğrenci tanıtım kartı

- Kullanıcıdan ad soyad, öğrenci numarası, sınıf ve GNO alın
- Sınıf ve GNO için TryParse kullanın; başarı değerlerini saklayın
- Bilgileri başlıklı ve hizalı bir kart olarak ekrana yazın
- GNO'yu iki ondalık basamakla gösterin
- En az üç test girdisi deneyip sonuçları not edin

Hafta sınırı

if/else kullanmak zorunlu değil. Bu hafta dönüşüm sonucunu gözlemliyoruz; karar verme 2. hafta.

Kod iskeleti

```
Console.Write("Ad Soyad: " );
string ad = Console.ReadLine() ?? "Misafir";

Console.Write("Öğrenci No: " );
string no = Console.ReadLine() ?? "-";

Console.Write("Sınıf: " );
bool sinifOk = int.TryParse(Console.ReadLine(), out int sinif);

Console.Write("GNO: " );
bool gnoOk = double.TryParse(Console.ReadLine(), out double gno);
```

Öğrenci tamamlar

Başlık çizgileri
Kart çıktısı
sinifOk ve gnoOk değerleri
Biçimlendirilmiş GNO

+1 LAB | TEST

Beklenen çıktı ve test matrisi

```
-------------------------
ÖĞRENCİ BİLGİ KARTI
Ad Soyad  : Ayşe Yılmaz
Öğrenci No: 20260042
Sınıf     : 1 (True)
GNO       : 3.18 (True)
-------------------------
```

Test 1	Geçerli tüm değerler
Test 2	Sınıf = bir • sinifOk ne olur?
Test 3	GNO = 3,18 ve 3.18 • kültür farkını gözlemle

Teslim kanıtı: program ekran çıktısı veya elle doldurulmuş çıktı tablosu.

Bugün kurduğumuz bilgi zinciri

Problem	İsimler ve fiiller
Model	Sınıf • nesne • durum • davranış
Program	C# kaynak kodu • .NET çalışma zamanı
Veri	Tür • değişken • değer
Etkileşim	Write/WriteLine • ReadLine • ??
Dönüşüm	TryParse • bool sonuç • out değer

HAFTA 1 TAMAMLANDI

Çıkış bileti ve gelecek haftaya köprü

- Sınıf ile nesne arasındaki farkı bir örnekle yazın.
- ReadLine neden doğrudan int değişkenine atanamaz?
- ?? operatörü hangi durumda sağdaki değeri kullanır?
- TryParse hangi iki sonucu üretir?
- Dikdörtgen uygulamasında negatif kenarı nasıl engellerdik?

Gelecek hafta

if / else
Karşılaştırma operatörleri
Mantıksal operatörler
Geçersiz girdiye tepki verme

Temel kaynaklar: yüklenen ders notları • Farrell Bölüm 1-2 • Albahari Bölüm 2 • Microsoft Learn